

VAE vs DDPM: Implementation from Scratch and Benchmark on CIFAR-10

Task code GenCV003 · Position: CV Engineer

1. Summary

I implemented a Variational Autoencoder (VAE) and a Denoising Diffusion Probabilistic Model (DDPM) from scratch in PyTorch (model, loss, training loop and sampler written by hand; no diffusers or pre-built VAE code), trained both on CIFAR-10 for 50 epochs, and compared them quantitatively (FID, Inception Score, training time, sampling speed) and qualitatively. Both models received the same number of gradient steps (19,500).

The DDPM produces far better samples: its FID is about 4× lower (32.2 vs 127.3) and its Inception Score about 2× higher (6.77 vs 3.46). The price is speed: the VAE generates an image with a single decoder pass, while the DDPM needs 1000 network evaluations, making it roughly 15,000× slower at sampling in my measurements and about 5.8× slower to train.

Model	FID ↓	Inception Score ↑	Train time	Sampling speed	Params
Real test images (reference)	3.15	10.96 ± 0.23	—	—	—
VAE	127.32	3.46 ± 0.10	14.3 min	≈ 14,000 img/s	2.9 M
DDPM (EMA weights)	32.22	6.77 ± 0.34	83.5 min	≈ 0.95 img/s	9.0 M

FID and IS use N = 3000 generated images per model against the 50,000 CIFAR-10 training images (see Sections 5 and 8 for caveats). Training on one NVIDIA T4 (Google Colab).

2. Experimental setup

- **Dataset:** CIFAR-10 (50,000 training images, 32×32 RGB). Training uses random horizontal flips; pixel values are scaled to [−1, 1]. Both models use the same data pipeline.
- **Framework:** PyTorch. Only the metrics (FID, IS) use an external library (torch-fidelity).
- **Hardware:** Google Colab, one NVIDIA T4 GPU. The DDPM was trained with mixed precision (AMP).
- **Reproducibility:** fixed seed (42), YAML configs per model, resumable checkpoints (saved to Google Drive), and a README with the exact commands.

3. VAE implementation

A VAE learns an encoder $q_\phi(z|x)$ and a decoder $p_\theta(x|z)$ by maximising the evidence lower bound (ELBO). With a diagonal-Gaussian posterior and a standard-normal prior the training loss is:

$$L = \|x - \hat{x}\|^2 \text{ (summed over pixels)} + \beta \cdot KL(q(z|x) \parallel N(0, I)), \beta = 1$$

Gradients flow through the sampling step thanks to the reparameterization trick: $z = \mu + \sigma \odot \epsilon$ with $\epsilon \sim N(0, I)$. At generation time the encoder is discarded: $z \sim N(0, I)$ is passed through the decoder once.

Component	Details
Encoder	3 strided convolutions (4×4, stride 2): 3→64→128→256 channels, BatchNorm + LeakyReLU(0.2); 32×32 → 4×4; flatten (4096) → two linear heads for μ and $\log \sigma^2$
Latent	128 dimensions

Component	Details
Decoder	linear 128 → 4096, reshape to 256×4×4, 3 transposed convolutions 256→128→64→3 with BatchNorm + ReLU, final Tanh (output in [-1, 1])
Loss	MSE summed over pixels + KL, averaged over the batch ($\beta = 1$)
Optimisation	Adam, lr 1e-3, batch 128, 50 epochs (19,500 steps)
Size / cost	2.9 M parameters; 14.3 min on a T4

Final training loss was 180.3 per image (reconstruction 113.3 + KL 67.0). The KL term grows slowly from 60 to 67 while the reconstruction term falls, i.e. the model gradually uses more of its latent capacity; the curve is nearly flat after epoch 40.

4. DDPM implementation

A DDPM defines a fixed forward process that gradually adds Gaussian noise to an image over $T = 1000$ steps with a linear schedule β_t from 1e-4 to 0.02. With $\alpha_t = 1 - \beta_t$ and $\bar{\alpha}_t = \prod \alpha_s$, any noisy version can be sampled in closed form:

$$x_t = \sqrt{\bar{\alpha}_t} \cdot x_0 + \sqrt{(1 - \bar{\alpha}_t)} \cdot \varepsilon, \varepsilon \sim N(0, I)$$

A U-Net $\varepsilon_\theta(x_t, t)$ is trained to predict the added noise (the “simple” loss of Ho et al.): $L = E_{t, x_0, \varepsilon} \|\varepsilon - \varepsilon_\theta(x_t, t)\|^2$. Generation starts from pure noise $x_T \sim N(0, I)$ and applies the learned reverse step 1000 times:

$$x_{t-1} = (1/\sqrt{\alpha_t}) \cdot (x_t - \beta_t/\sqrt{(1-\bar{\alpha}_t)} \cdot \varepsilon_\theta(x_t, t)) + \sigma_t z, \sigma_t^2 = \beta_t$$

Component	Details
U-Net	base width 64, channel multipliers (1, 2, 2, 2), 2 residual blocks per level, resolutions 32→16→8→4 with skip connections, self-attention at 16×16 and in the bottleneck, GroupNorm + SiLU, dropout 0.1
Time conditioning	sinusoidal embedding of $t \rightarrow$ 2-layer MLP → added inside every residual block
Training	Adam, lr 2e-4, batch 128, gradient clipping 1.0, mixed precision, 50 epochs (19,500 steps)
EMA	exponential moving average of weights (decay 0.999); all samples and metrics use the EMA model
Sampling	full ancestral sampling, 1000 steps, output clipped to [-1, 1]
Size / cost	8.95 M parameters; 83.5 min on a T4

The noise-prediction loss drops quickly (0.089 → 0.040 within 3 epochs) and then plateaus around 0.031. This plateau does not mean sample quality stops improving; the loss is a poor proxy for visual quality, which is why I judge the model by samples and FID.

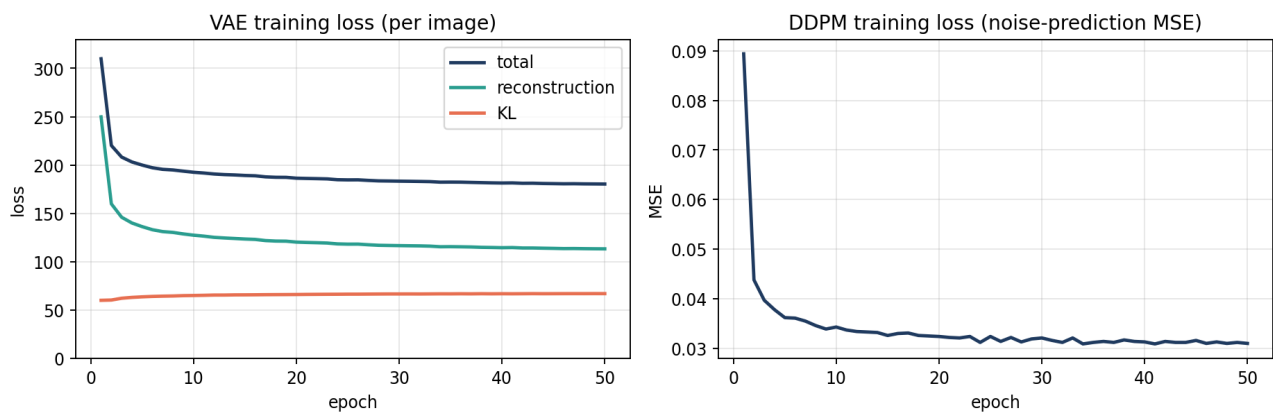


Figure 1. Training curves (per-epoch averages) for the VAE (left) and the DDPM (right).

5. Evaluation protocol

- **Samples:** $N = 3000$ images per model. VAE: $z \sim \mathcal{N}(0, I)$. DDPM: 1000-step ancestral sampling from the EMA model. The same N is used for both models.
- **FID and Inception Score:** computed with torch-fidelity against the 50,000 CIFAR-10 training images.
- **Sanity check:** the 10,000 real test images scored against the training set give FID 3.15 and IS 10.96, matching the usual values for CIFAR-10, which confirms the metric pipeline works.
- **Speed:** pure sampling time per image for a batch of 64 on the T4 (no image saving), fp32. Training time is the sum of epoch times recorded during training.

6. Quantitative results

Metric	VAE	DDPM	Ratio
FID (lower is better)	127.32	32.22	DDPM 3.95× lower
Inception Score (higher is better)	3.46 ± 0.10	6.77 ± 0.34	DDPM 1.96× higher
Training time (50 epochs)	14.3 min	83.5 min	DDPM 5.8× longer
Sampling time per image	$7.1e-5$ s	1.06 s	DDPM $\approx 15,000\times$ slower
Network evaluations per sample	1	1000	—
Parameters	2.9 M	9.0 M	DDPM 3.1× larger

The DDPM wins clearly on distribution quality, while the VAE wins by orders of magnitude on speed. Three points are worth noting when reading these numbers:

- **Fair step budget, unequal cost.** Both models saw 19,500 gradient steps, yet the DDPM took 5.8× longer because its U-Net is larger and uses attention.
- **Capacity is not matched.** The DDPM has 3.1× more parameters, so part of the gap may come from network size and not only from the modelling approach. I did not run a matched-capacity ablation.
- **Sample-size bias.** FID is biased upwards for small N . As a check, the same DDPM checkpoint scored FID 112 with only 250 images and 32.2 with 3000. The comparison between models is valid (same N), but absolute values should not be compared with published numbers, which use 50,000 samples.

7. Qualitative analysis



Figure 2. Left: real CIFAR-10 test images. Middle: 64 VAE samples. Right: 64 DDPM samples. The grids show the first 64 images generated with a fixed seed; they were not hand-picked.

Realism. VAE samples are smooth and blurry: they capture the global colour layout (a bright sky-like region on top, dark or green ground below, a central blob) but have no sharp edges, textures or recognisable objects. DDPM samples contain sharp edges and high-frequency texture; many are clearly recognisable as cars, ships, aircraft, horses or other animals, with plausible lighting and backgrounds. The DDPM is not perfect: animals often have distorted anatomy, some objects are fused with their background, and a few samples are hard to classify, consistent with the short 50-epoch training.

Diversity. Both grids show a wide range of colours and layouts, and I see no sign of mode collapse in either. The VAE's variety is mostly in colour and coarse shape, and its samples share a soft “averaged” look. The DDPM's diversity extends to object type, pose, background and texture, closer to the real-image grid. This judgement is visual only; I did not compute a diversity metric such as precision/recall.

Thematic consistency. DDPM samples follow the CIFAR-10 composition: a single dominant object near the centre in a class-typical setting (water around boats, sky around aircraft, grass or earth around animals, road around vehicles). VAE samples keep only the coarse colour statistics of such scenes, so the theme is hinted at but the object itself is not recognisable.

8. The main differences between the two approaches

VAE: one-step generation through a learned latent bottleneck. An encoder compresses each image into a low-dimensional latent code, and the decoder reconstructs it. Training maximises the ELBO. With a Gaussian decoder of fixed variance the reconstruction term becomes mean-squared error, which rewards the *average* of all images consistent with a latent code, so fine detail is smoothed out and samples look blurry. The KL term further pulls the posterior towards the prior, trading reconstruction detail for a latent space that can be sampled. Generation costs a single decoder pass.

DDPM: iterative denoising with no encoder. The forward (noising) process is fixed, not learned, and the network only learns to remove a small amount of noise at each of 1000 levels. The ELBO decomposes into per-step denoising terms, which reduce to a simple noise-prediction MSE. Because each step is a small, easy regression problem and the “latent” has the same size as the image, no information is squeezed through a bottleneck, so fine detail is preserved. The cost is that generating one image needs 1000 sequential network evaluations.

	VAE	DDPM
Latent / structure	Compact learned latent (128-d), encoder + decoder	No encoder; latent is image-sized noise, fixed forward process

	VAE	DDPM
Training objective	ELBO: reconstruction (MSE) + KL	Noise-prediction MSE (simplified ELBO)
Sampling	1 decoder pass	1000 sequential U-Net evaluations
Sample quality (this study)	Blurry, FID 127	Sharp, recognisable, FID 32
Speed (this study)	$\approx 14,000$ img/s	≈ 0.95 img/s
Main failure mode	Blur from averaging; prior/posterior mismatch	Slow sampling; needs longer training for clean details

In short: the VAE trades quality for speed and a useful latent space; the DDPM trades speed for quality. The speed gap can be reduced with faster samplers (e.g. DDIM) or by running diffusion in a VAE's latent space, and the blur of the VAE can be reduced with stronger decoders or different likelihoods, but these were outside the scope of this task.

9. Limitations and possible improvements

- **Short DDPM training:** 50 epochs ($\approx 19.5k$ steps), far less than the original paper's 800k steps; I expect the FID to keep improving with longer training (the 100-epoch run was stopped early because of time).
- **N = 3000 samples** for FID/IS (biased upwards; use 10k-50k for publishable numbers), a single run and a single seed, so no confidence intervals.
- **Different capacities** (2.9 M vs 9.0 M parameters) and no hyper-parameter search for either model; the VAE used $\beta = 1$ and MSE only.
- **Metric implementation:** FID/IS come from torch-fidelity; values can differ slightly from other implementations. The real-image reference score was computed on 10,000 images, not 3000.
- **Speed numbers** were measured on a T4 with a batch of 64 and depend on batch size and hardware; the VAE figure in particular is batch-dependent.
- **Next steps:** longer DDPM training, β -VAE ablation, DDIM sampling with fewer steps, larger N, multiple seeds, matched-capacity comparison, and a precision/recall metric for diversity.

10. Conclusion

Both generative models were implemented from scratch and trained under identical conditions. The experiment reproduces the well-known trade-off: the VAE is cheap and extremely fast but produces blurry images (FID 127.3, IS 3.46), whereas the DDPM produces much sharper and more diverse images (FID 32.2, IS 6.77) at the cost of 1000-step sampling that is about 15,000 $\times$ slower. The fundamental difference lies in how each model turns noise into images: a single learned decoding through a latent bottleneck versus many small learned denoising steps.

Code and instructions to reproduce are in the GitHub repository (see the README: environment setup, training commands for both models, evaluation command, and Colab notebooks).